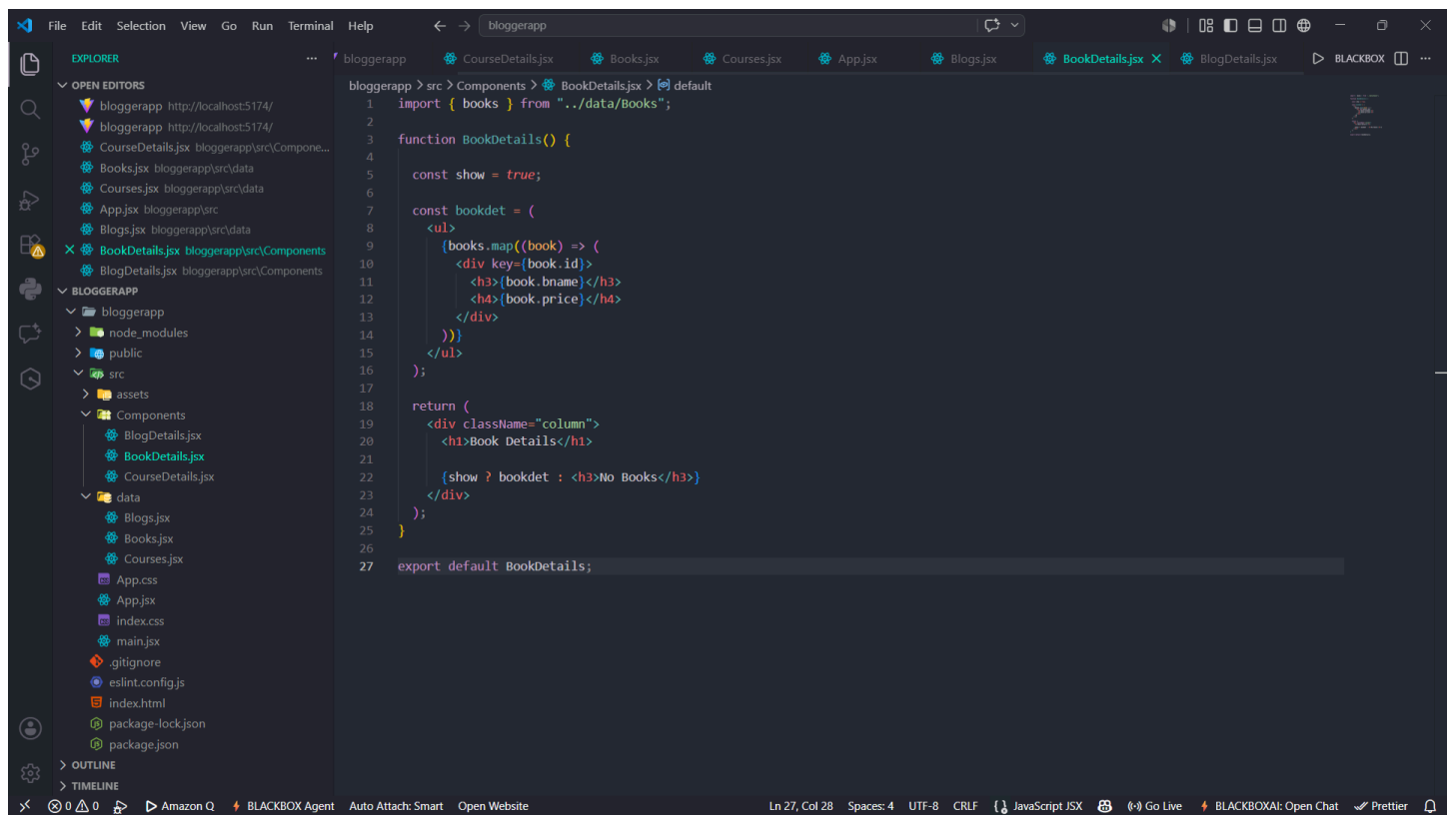


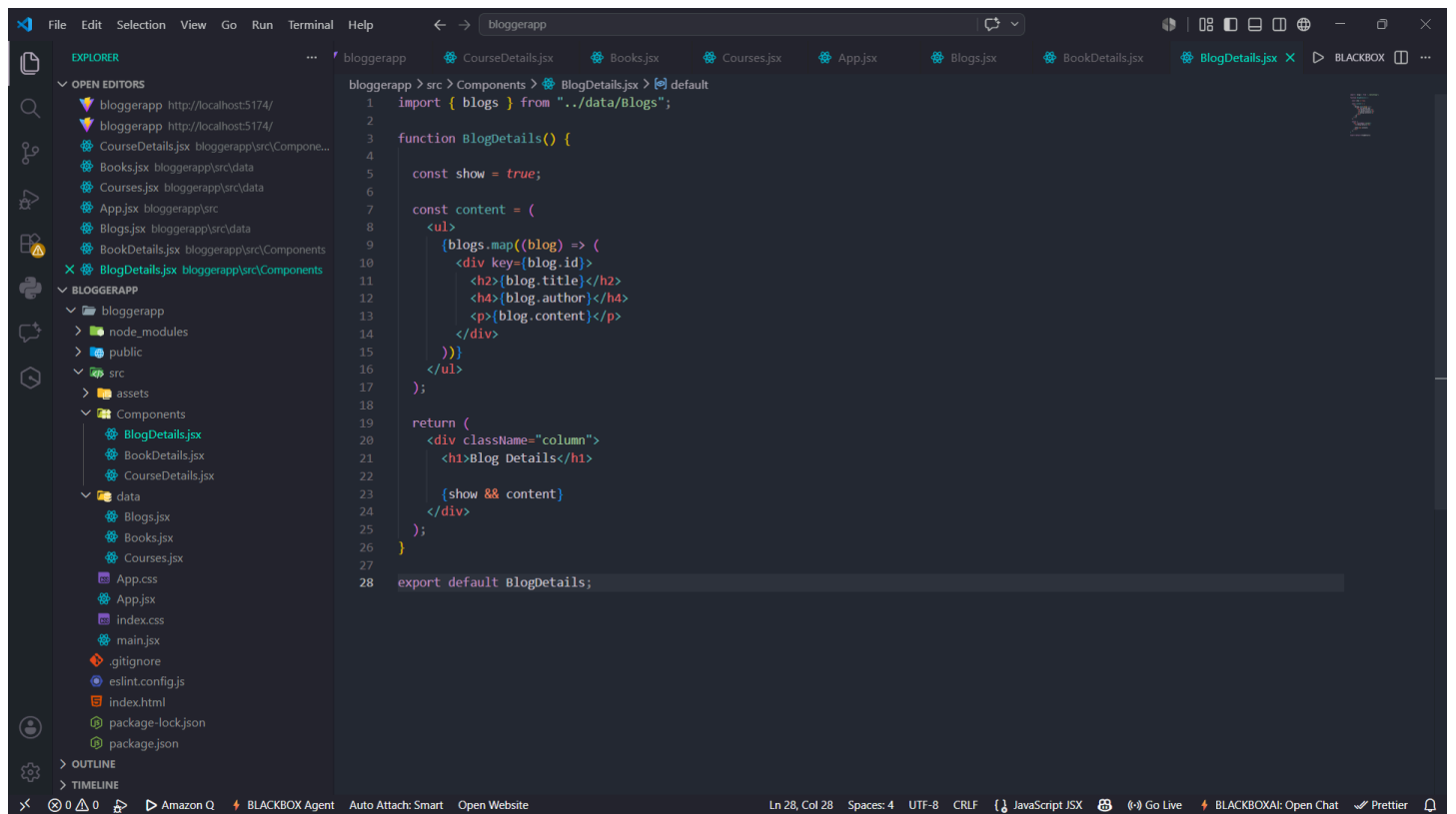
BookDetails.jsx



The screenshot shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with folders like 'src', 'components', and 'data'. The code editor displays the 'BookDetails.jsx' file, which is a React component. The code imports 'books' from '../data/Books' and defines a 'BookDetails' function. The function has a 'show' prop defaulting to 'true' and a 'bookdet' prop. It uses 'books.map' to render a list of books, each with a 'key' of 'book.id'. The rendered HTML includes a 'div' with 'className="column"', a 'h1' for 'Book Details', and a conditional rendering of 'No Books' if 'show' is false. The component is exported as 'BookDetails'.

```
1 import { books } from "../data/Books";
2
3 function BookDetails() {
4
5     const show = true;
6
7     const bookdet = (
8         <ul>
9             {books.map((book) => (
10                 <div key={book.id}>
11                     <h3>{book.bname}</h3>
12                     <h4>{book.price}</h4>
13                 </div>
14             ))}
15         </ul>
16     );
17
18     return (
19         <div className="column">
20             <h1>Book Details</h1>
21             {show ? bookdet : <h3>No Books</h3>}
22         </div>
23     );
24 }
25
26 export default BookDetails;
```

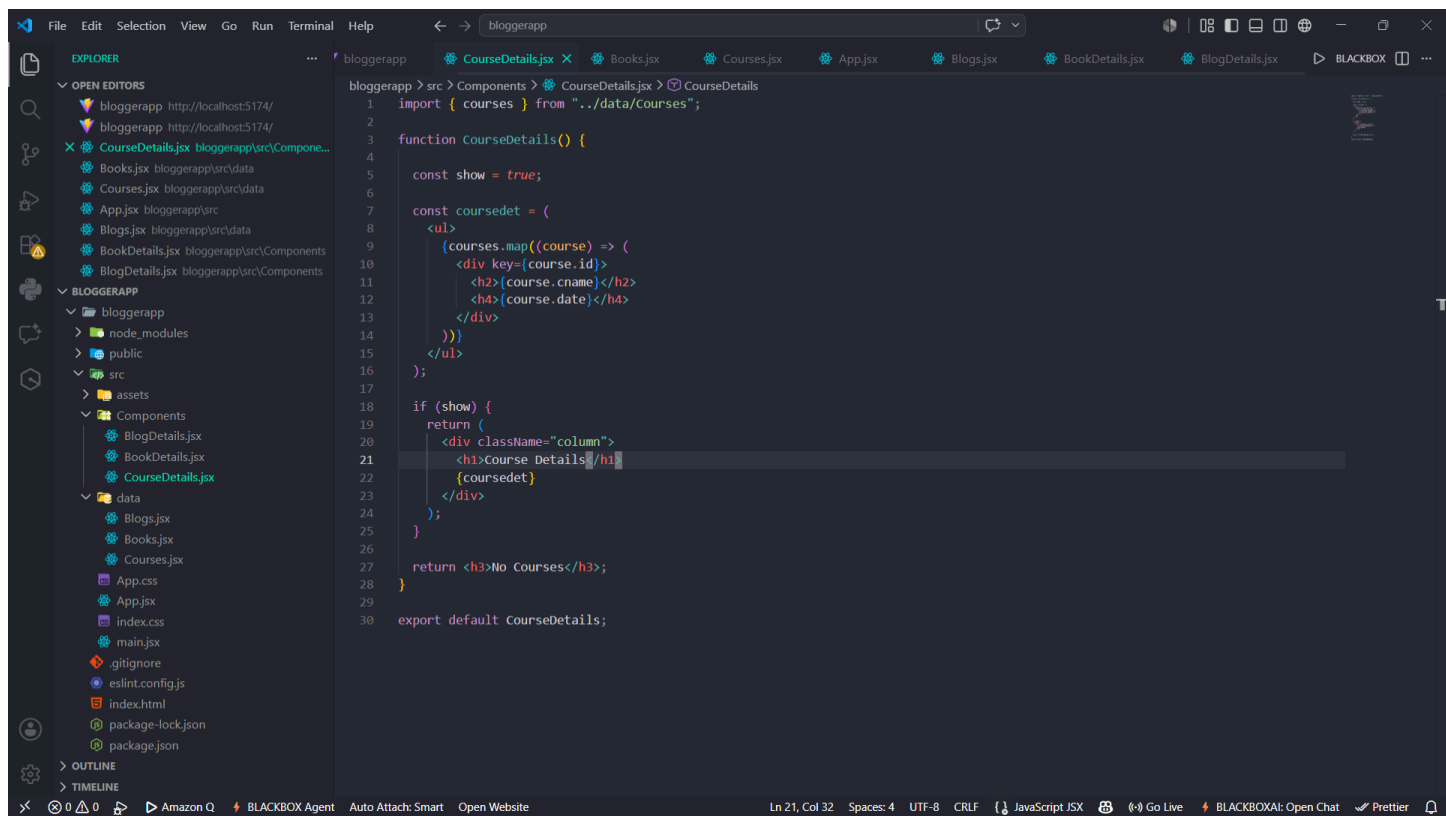
Blog details.jsx



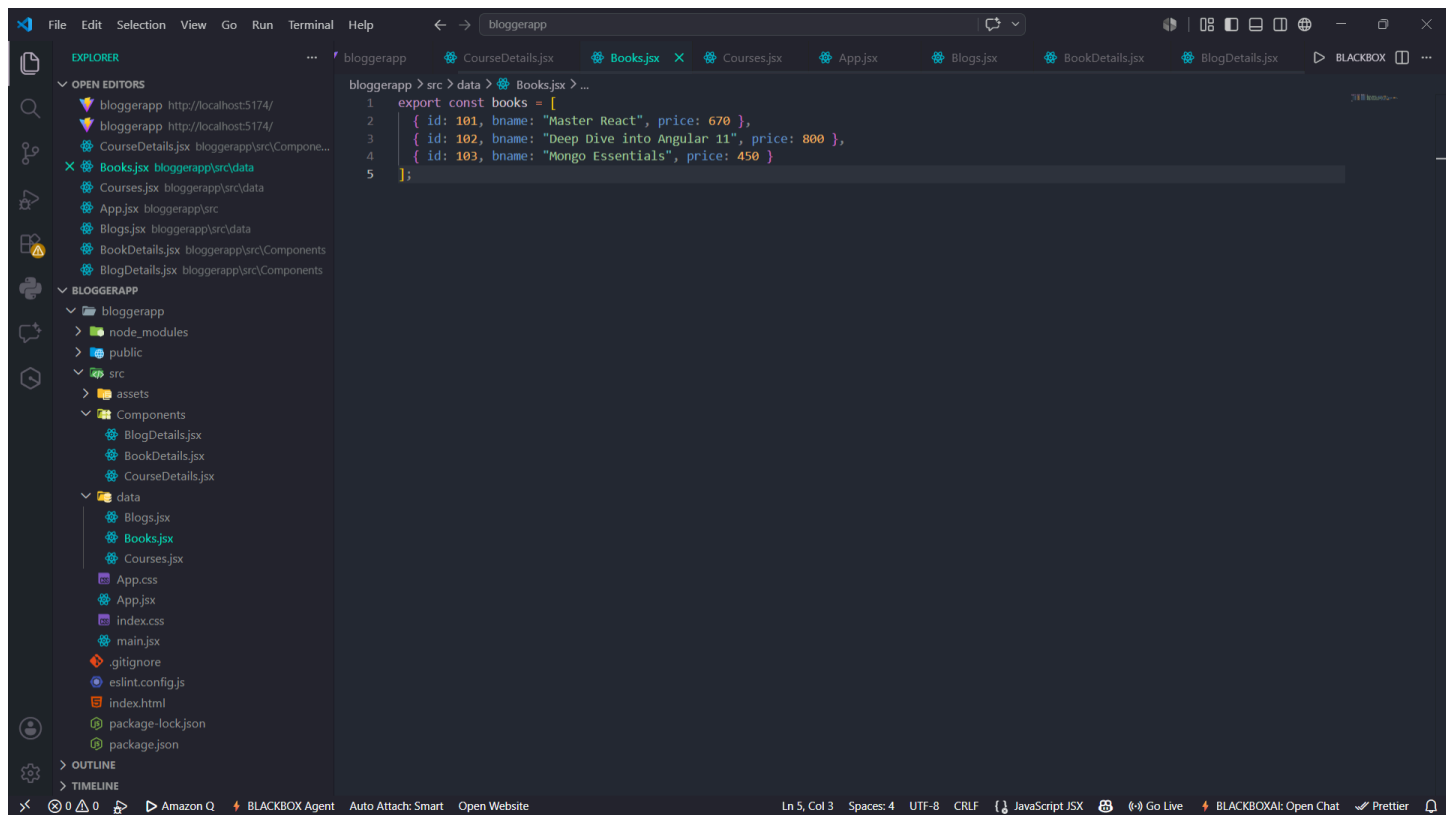
The screenshot shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with folders like 'src', 'components', and 'data'. The code editor displays the 'BlogDetails.jsx' file, which is a React component. The code imports 'blogs' from '../data/Blogs' and defines a 'BlogDetails' function. The function has a 'show' prop defaulting to 'true' and a 'content' prop. It uses 'blogs.map' to render a list of blogs, each with a 'key' of 'blog.id'. The rendered HTML includes a 'div' with 'className="column"', a 'h1' for 'Blog Details', and a conditional rendering of 'show && content' if 'show' is true. The component is exported as 'BlogDetails'.

```
1 import { blogs } from "../data/Blogs";
2
3 function BlogDetails() {
4
5     const show = true;
6
7     const content = (
8         <ul>
9             {blogs.map((blog) => (
10                 <div key={blog.id}>
11                     <h2>{blog.title}</h2>
12                     <h4>{blog.author}</h4>
13                     <p>{blog.content}</p>
14                 </div>
15             ))}
16         </ul>
17     );
18
19     return (
20         <div className="column">
21             <h1>Blog Details</h1>
22             {show && content}
23         </div>
24     );
25 }
26
27 export default BlogDetails;
```

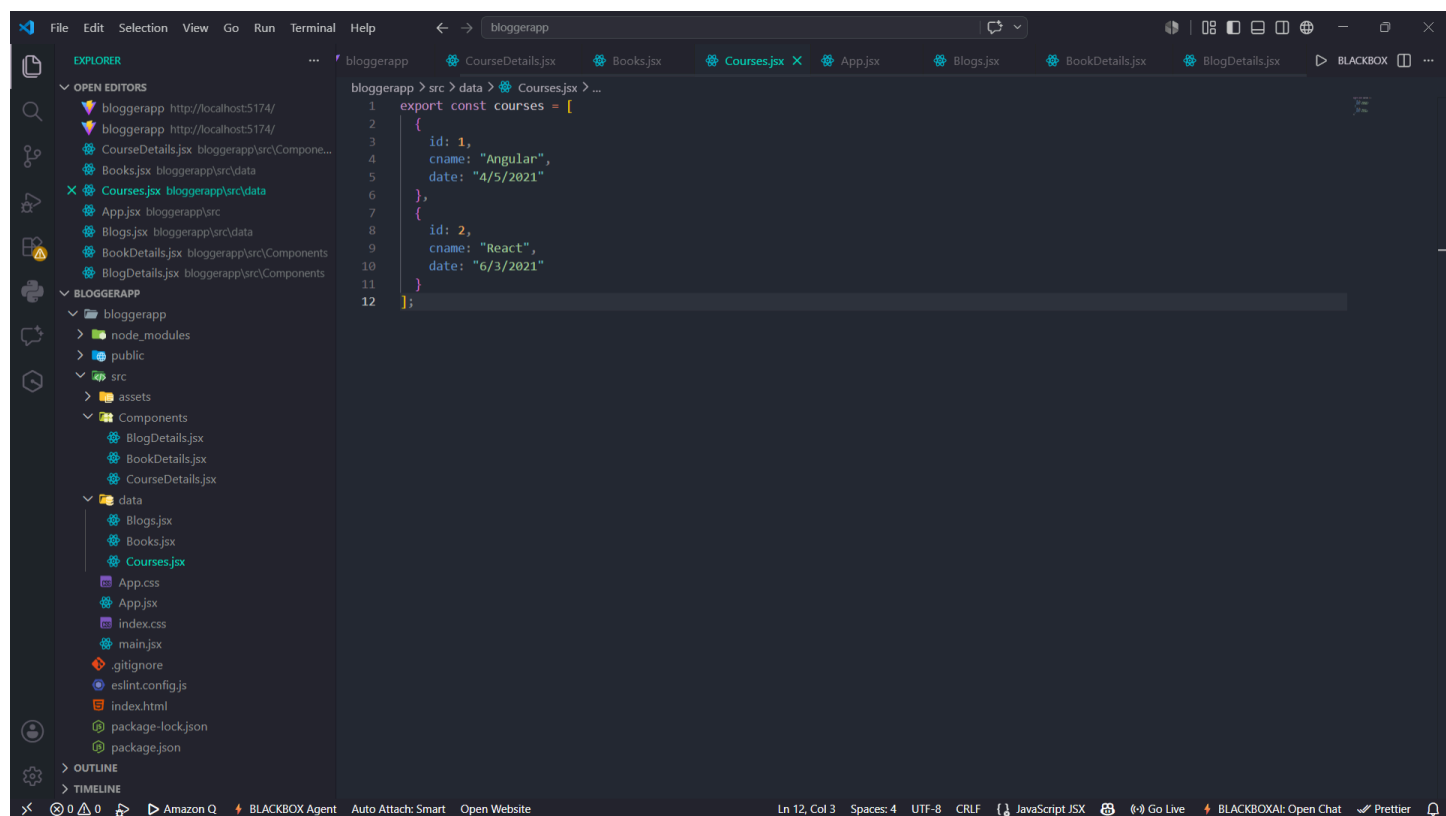
coursedetails.jsx



Books.jsx



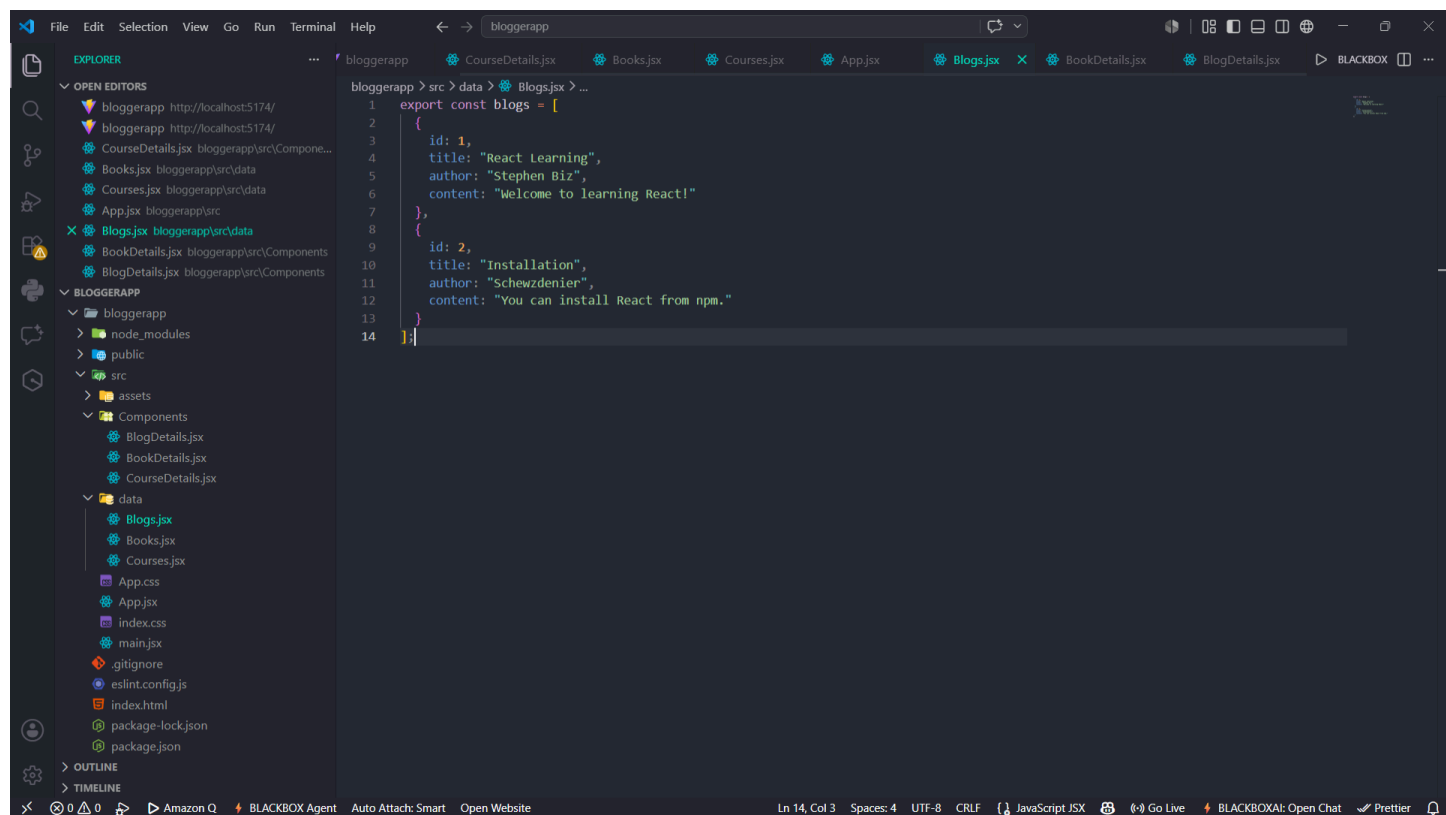
course.jsx



The screenshot shows the VS Code editor interface for the 'bloggerapp' project. The Explorer sidebar on the left displays the project structure, including folders like 'src', 'components', and 'data', and files like 'App.jsx', 'App.css', 'index.css', 'main.jsx', '.gitignore', 'eslint.config.js', 'index.html', 'package-lock.json', and 'package.json'. The 'data' folder is expanded, showing 'Blogs.jsx', 'Books.jsx', and 'Courses.jsx'. The 'Courses.jsx' file is selected and its content is displayed in the editor. The status bar at the bottom indicates the file is at line 12, column 3, using UTF-8 encoding with CRLF line endings.

```
1 export const courses = [
2   {
3     id: 1,
4     cname: "Angular",
5     date: "4/5/2021"
6   },
7   {
8     id: 2,
9     cname: "React",
10    date: "6/3/2021"
11  }
12 ];
```

Blogs.jsx



The screenshot shows the VS Code editor interface for the 'bloggerapp' project, with the 'Blogs.jsx' file selected in the Explorer sidebar. The file's content is displayed in the editor. The status bar at the bottom indicates the file is at line 14, column 3, using UTF-8 encoding with CRLF line endings.

```
1 export const blogs = [
2   {
3     id: 1,
4     title: "React Learning",
5     author: "Stephen Biz",
6     content: "Welcome to learning React!"
7   },
8   {
9     id: 2,
10    title: "Installation",
11    author: "Schewzdenier",
12    content: "You can install React from npm."
13  }
14 ];
```

Objectives

1. Explain various ways of Conditional Rendering

Conditional rendering is the process of displaying different UI elements based on a condition.

- **if...else:** Renders different components based on a condition.
- **Ternary Operator (? :):** Displays one element if true and another if false.
- **Logical AND (&&):** Renders an element only when the condition is true.
- **Switch Statement:** Used when multiple conditions need to be checked.
- **Element Variables:** Store JSX in variables and render them based on conditions.

2. Explain how to render multiple components

Multiple components are rendered by importing them into a parent component and placing them inside its JSX. This helps organize the application into reusable and manageable parts.

3. Define List Component

A **List Component** is a component that displays a collection of items from an array. It is commonly created using the `map()` function to generate UI elements dynamically.

4. Explain about Keys in React Applications

A **key** is a unique identifier given to list items. It helps React identify which items have changed, been added, or removed, making updates faster and more efficient.

5. Explain how to extract components with keys

When a list item is moved into a separate reusable component, the **key** should be assigned in the parent component where the list is created, not inside the child component.

6. Explain React `map()` Function

The **`map()`** function is a JavaScript array method used in React to convert each item in an array into a JSX element. It is the most common way to render lists dynamically.